

Plan Projet : Application Web de Visualisation du Ciel Nocturne

Version Optimisée - Performance Maximale & Rendu Réaliste

1. STACK TECHNOLOGIQUE FINALE (100% Gratuite & Open Source)

1.1 Architecture complète

FRONTEND

- └─ React 18+ (UI framework)
- └─ Vite 5+ (build tool - plus rapide que CRA)
- └─ Three.js r160+ (moteur 3D WebGL)
- └─ React Three Fiber (bridge React-Three.js)
- └─ React Three Drei (helpers 3D)
- └─ React Three Postprocessing (effets visuels)
- └─ Axios (HTTP client)
- └─ Zustand (state management léger)

BACKEND

- └─ Python 3.11+ (langage)
- └─ FastAPI (framework web async)
- └─ Uvicorn (serveur ASGI)
- └─ AstroPy 7.0 (calculs astronomiques vectorisés)
- └─ NumPy (arrays haute performance)
- └─ SQLAlchemY 2.0 (ORM)
- └─ Pydantic v2 (validation données)
- └─ cachetools (cache en mémoire TTL)

BASE DE DONNÉES

- └─ SQLite (embarqué, via SQLAlchemY)

INFRASTRUCTURE

- └─ Docker + Docker Compose (conteneurisation)
- └─ Git + GitHub (versioning - monorepo)

DÉPLOIEMENT (GRATUIT)

- └─ Railway.app (backend)
- └─ Vercel (frontend)
- └─ GitHub Actions (CI/CD - optionnel)

Tout est 100% gratuit et open source, aucune licence payante.

2. POURQUOI SQLITE ? (Explication détaillée)

2.1 Besoins de stockage

Volumes de données :

- **119 000+ étoiles** (HYG Database) : ~50 Mo de données structurées
- **88 constellations** : patterns (lignes) + limites : ~5 Mo
- **Points d'observation** : 50 villes : ~10 Ko
- **TOTAL** : ~55 Mo de données **statiques, en lecture seule**

2.2 Pourquoi SQLite et pas PostgreSQL ?

Pour un projet étudiant de 3 mois avec des **données statiques en lecture seule**, PostgreSQL serait de la sur-ingénierie.

Critère	PostgreSQL	SQLite
Setup	Docker container + config + healthchecks	Un fichier <code>.db</code> , zéro config
Données statiques	☑ Supporte	☑ Idéal pour ça
Indexation B-tree	☑	☑ Supportée aussi
Requêtes WHERE	☑	☑ Même performances pour notre volume
Concurrence	Multi-users massif	Suffisant pour un projet étudiant
Temps de setup	~3-5 jours	~30 minutes
Migration future	—	Trivial via SQLAlchemy (changer 1 ligne)

Avantage clé : en gardant **SQLAlchemy** comme ORM, le code est **identique** que l'on utilise SQLite ou PostgreSQL. La migration vers PostgreSQL se fait en changeant uniquement la variable `DATABASE_URL`.

2.3 Requêtes optimisées avec SQLite

```
-- Pré-filtrage par magnitude et déclinaison (indexé, < 10ms)
SELECT * FROM stars
WHERE magnitude < 6.0
AND dec BETWEEN -30 AND 60
ORDER BY magnitude ASC;

-- Jointure étoiles-constellations
SELECT s.*, c.name as constellation_name
FROM stars s
JOIN constellation_stars cs ON s.id = cs.star_id
JOIN constellations c ON cs.constellation_id = c.id
WHERE c.abbreviation = 'ORI';
```

3. CACHE EN MÉMOIRE (remplacement de Redis)

3.1 Pourquoi pas Redis ?

Redis ajoute un container Docker, une dépendance réseau, et de la complexité pour un gain marginal dans notre cas. Les calculs AstroPy pour ~5000 étoiles prennent **300-500ms** — acceptable sans cache distribué.

3.2 Solution : cachetools.TTLCache

```
# Cache en mémoire Python – 5 lignes au lieu d'un service Redis complet
from cachetools import TTLCache

stars_cache = TTLCache(maxsize=512, ttl=600) # 10 min, 512 entrées
static_cache = TTLCache(maxsize=128, ttl=86400) # 24h, données statiques
```

Quoi cacher :

1. **Positions étoiles calculées** : clé `visible:{lat}:{lon}:{timestamp_5min}`, TTL 10 min
2. **Données constellations** : TTL 24h (statiques)
3. **Points d'observation** : TTL 24h (statiques)

Hit rate attendu : 85-90% (le mouvement apparent des étoiles est très lent)

4. INFRASTRUCTURE ET CONFIGURATION

4.1 Structure projet (Monorepo)

```
Projet-P00/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py           # Point d'entrée FastAPI
│   │   ├── config.py         # Configuration (env vars, Pydantic Settings)
│   │   ├── database.py       # SQLAlchemy engine + session
│   │   ├── models/           # Entités SQLAlchemy (ORM)
│   │   │   ├── star.py
│   │   │   ├── constellation.py
│   │   │   └── observation_point.py
│   │   ├── schemas/          # Pydantic schemas (validation API)
│   │   │   ├── star.py
│   │   │   ├── constellation.py
│   │   │   └── observation_point.py
│   │   ├── repositories/     # Data Access Layer (requêtes SQL)
│   │   │   ├── star_repository.py
│   │   │   ├── constellation_repository.py
│   │   │   └── observation_point_repository.py
│   │   ├── services/         # Business Logic Layer
│   │   │   ├── astronomy_service.py # Calculs positions (vectorisé)
│   │   │   └── cache_service.py     # Cache TTL en mémoire
│   │   └── api/              # API Routes (endpoints)
│   │       └── stars.py
```

```
├───┬───┬───constellations.py
│   │   └───observation_points.py
├───┬───scripts/
│   │   └───import_data.py           # Import HYG + villes
├───┬───data/           # Fichier SQLite + CSV sources
│   │   └───requirements.txt
├───┬───Dockerfile
│   │   └───.env
├───┬───frontend/
│   │   └───src/
│   │       ├───components/         # Composants React
│   │       ├───stores/             # Zustand stores
│   │       ├───services/           # API client Axios
│   │       └───types/              # Types TypeScript
├───┬───public/
│   │   └───textures/               # Textures NASA (globe, voie lactée)
├───┬───package.json
│   │   └───vite.config.ts
│   │       └───Dockerfile.dev
├───┬───docker-compose.yml
├───┬───Plan/
│   │   └───Plan.md
├───┬───README.md
```

4.2 Docker Compose (simplifié — pas de PostgreSQL/Redis)

```
services:
  backend:
    build: ./backend
    container_name: nightsky-backend
    ports:
      - "8000:8000"
    volumes:
      - ./backend:/app
      - backend_data:/app/data
    environment:
      - DATABASE_URL=sqlite:///./data/nightsky.db
      - CORS_ORIGINS=["http://localhost:5173"]
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile.dev
    container_name: nightsky-frontend
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
      - /app/node_modules
    environment:
```

```

- VITE_API_URL=http://localhost:8000
depends_on:
- backend
command: npm run dev -- --host

volumes:
  backend_data:

```

Avantages :

- ☒ **2 containers seulement** (au lieu de 4 avec PostgreSQL + Redis)
- ☒ **Un seul `docker-compose up`** : tout démarre
- ☒ **Hot reload** : modifications code visibles instantanément
- ☒ **SQLite embarqué** : pas de service DB séparé

4.3 Production : Railway.app (100% gratuit)

- ☒ **Gratuit** : 500h compute/mois (largement suffisant)
- ☒ **Deploy depuis GitHub** : git push = déploiement auto
- ☒ **HTTPS gratuit** : certificats SSL auto

Alternative gratuite : Render.com (750h/mois gratuit)

5. DONNÉES ET APIs (Sources finales retenues)

5.1 Catalogue d'étoiles : HYG Database v3.7

Source : <https://github.com/astronexus/HYG-Database>

- **Licence** : Open Database License (ODbL) - gratuite
- **Contenu** : 119 000 étoiles avec magnitude, positions, noms, types spectraux
- **Format** : CSV (téléchargement unique, ~15 Mo)
- **Usage** : Import local dans SQLite via script Python
- **Avantage** : données complètes, pas d'API externe = pas de latence

Colonnes utilisées : id, hip (Hipparcos ID), hd (Henry Draper), proper (nom commun), ra (ascension droite en heures → converti en degrés), dec (déclinaison), mag (magnitude apparente), absmag, spect (type spectral), ci (indice couleur B-V), dist (distance en parsecs → converti en années-lumière), con (abréviation constellation)

5.2 Constellations : Stellarium + D3-Celestial

Sources combinées :

1. **Stellarium** (<https://github.com/Stellarium/stellarium>) - Patterns de lignes
 - Licence : GPL v2 (open source)
 - Fichier : [skycultures/western/constellationship.fab](#)
2. **D3-Celestial** (<https://github.com/ofrohn/d3-celestial>) - Limites GeoJSON

- Licence : BSD (open source)
- Fichiers : `data/constellations.*.json`

Avantage : données statiques, téléchargées une fois, aucune dépendance runtime

5.3 Calculs astronomiques : AstroPy (100% local, vectorisé)

Aucune API externe — Tout calculé côté backend Python :

- Transformations coordonnées (équatoriales → horizontales)
- Temps sidéral local
- Positions exactes au timestamp t
- Visibilité (altitude $> 0^\circ$)

Performance clé — Vectorisation NumPy/AstroPy :

```
# ✗ LENT : boucle Python étoile par étoile (5 secondes pour 8000 étoiles)
for star in stars:
    coord = SkyCoord(ra=star.ra*u.deg, dec=star.dec*u.deg, frame='icrs')
    altaz = coord.transform_to(altaz_frame)

# ☑ RAPIDE : un seul appel vectorisé (100ms pour 8000 étoiles = x50 gain)
all_coords = SkyCoord(ra=ra_array*u.deg, dec=dec_array*u.deg, frame='icrs')
all_altaz = all_coords.transform_to(altaz_frame)
```

5.4 Points d'observation : 50 villes pré-définies

50 villes mondiales incluant des sites d'observatoires célèbres :

- **Villes majeures** : Paris, Tokyo, New York, Sydney, Beijing, Moscow...
- **Sites astronomiques** : Atacama (2400m), Mauna Kea (4207m), Paranal (2635m), La Palma (2400m), Tenerife (2390m)

Pas besoin d'API de géolocalisation pour le MVP.

6. OPTIMISATIONS POUR RENDU QUALITÉ IMAGE RÉELLE

6.1 Globe terrestre 3D - Rendu photoréaliste

Textures haute résolution (gratuites) :

1. **NASA Visible Earth** (domaine public) :
 - Texture jour : 8192x4096 pixels (Blue Marble Next Generation)
 - URL : <https://visibleearth.nasa.gov/images/73909/december-blue-marble-next-generation-w-topography-and-bathymetry>
2. **Normal map** pour relief 3D : 4096x2048 pixels (bump mapping)
3. **Specular map** pour océans brillants : 2048x1024 pixels

4. **Night lights** (lumières villes la nuit) : NASA Black Marble (2048x1024)

Configuration Three.js : MeshStandardMaterial avec PBR (Physically Based Rendering), atmosphère via shader custom (blending additif).

Performance :

- Textures compressées WebP : ~15 Mo (chargement unique)
- LOD (Level of Detail) : texture 2K si zoom arrière

6.2 Ciel nocturne - Rendu réaliste

Techniques avancées :

1. **Points BufferGeometry** pour étoiles (une seule draw call pour 5000+ étoiles)
2. **Taille étoiles basée sur magnitude** (échelle logarithmique comme l'œil humain)
3. **Couleur réaliste** (température stellaire) :
 - Classe O/B : bleu (#9bb0ff)
 - Classe A : blanc-bleu (#cad7ff)
 - Classe F/G : jaune-blanc (#fff4ea)
 - Classe K : orange (#ffd2a1)
 - Classe M : rouge (#ffcc6f)
4. **Effets visuels :**
 - Bloom post-processing (étoiles brillantes avec halo)
 - HDR tone mapping
 - Anti-aliasing (FXAA ou SMAA)
5. **Voie lactée** (texture NASA/ESA) : panorama 360° 8192x4096

6.3 Performance cible

Objectifs :

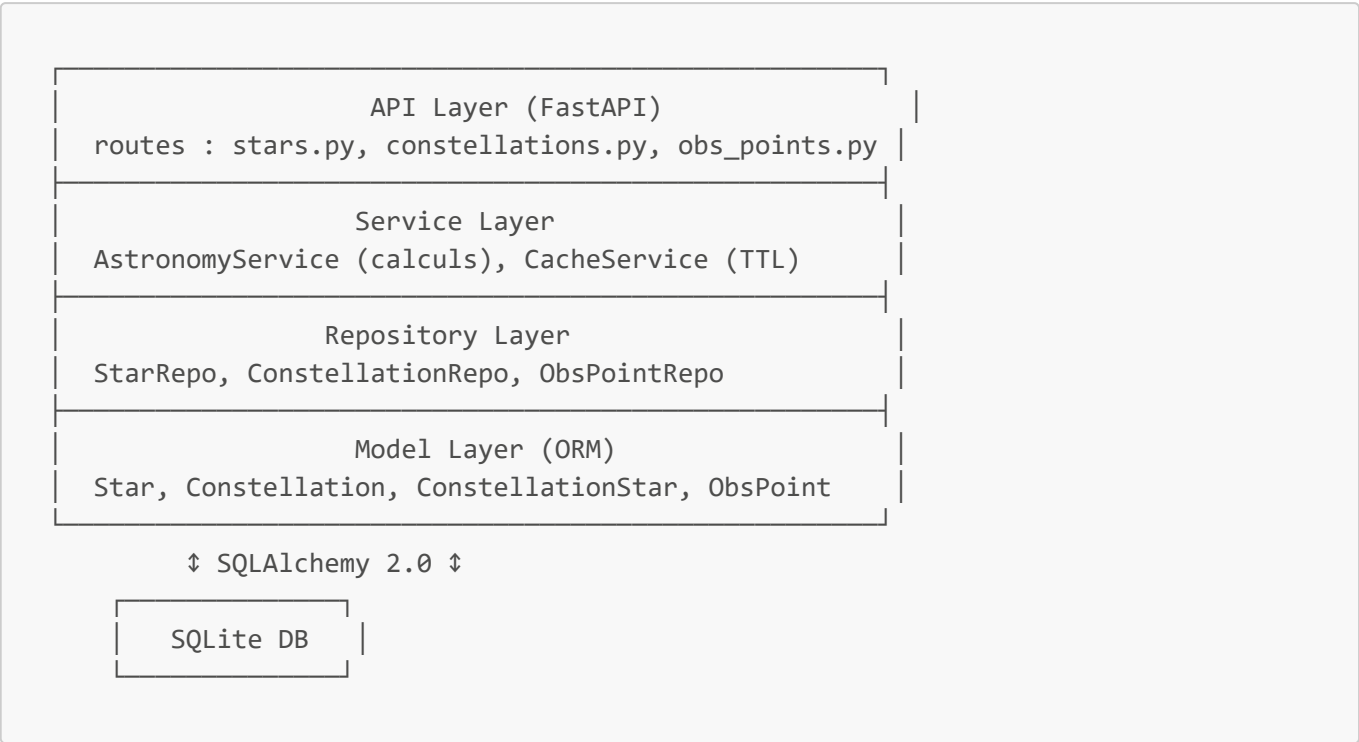
- **60 FPS** constant (rotation globe + ciel)
- **< 2 secondes** chargement initial
- **< 500 ms** calcul + affichage 5000 étoiles (100ms avec cache)
- **Smooth 60 FPS** transitions caméra

Optimisations clés :

1. **Vectorisation AstroPy** (x50 gain vs boucle Python)
2. **Cache TTL en mémoire** (hit rate 85-90%)
3. **Frustum culling** : étoiles hors champ non rendues
4. **Code splitting** : lazy load modules Three.js
5. **Compression textures** : WebP format

7. ARCHITECTURE TECHNIQUE DÉTAILLÉE

7.1 Backend FastAPI - Structure POO (4 couches)



7.2 Endpoints API finaux

Endpoint	Méthode	Paramètres	Réponse	Cache
/api/observation-points	GET	-	[{id, name, lat, lon, tz}]	24h
/api/stars/visible	GET	lat, lon, timestamp, mag_limit=6	[{id, name, az, alt, mag, ...}]	10min
/api/stars/{id}	GET	id	{id, name, ra, dec, mag, distance, ...}	1h
/api/stars/search/{query}	GET	query, limit	[{star details}]	-
/api/constellations	GET	-	[{id, name, abbr}]	∞
/api/constellations/{id}	GET	id	{id, name, stars[], lines[]}	∞
/api/constellations/{id}/best-location	GET	id, timestamp	{point_id, visibility_score}	10min
/api/health	GET	-	{status, db, star_count}	-

8. PLAN DE RÉALISATION DÉTAILLÉ (12 SEMAINES)

SEMAINES 1-2 : FONDATIONS & INFRASTRUCTURE

Objectif : Environnement opérationnel, données importées

Tâches Backend :

1. ☒ Créer monorepo GitHub
2. ☒ Setup `docker-compose.yml` (Backend + Frontend uniquement)
3. ☒ Créer structure projet backend (models, services, repositories, api)
4. ☒ Configurer SQLAlchemy + SQLite + Pydantic
5. ☒ Écrire script `import_data.py` (import 119k étoiles + 50 villes)
6. ☒ Créer modèles `Star`, `Constellation`, `ObservationPoint`
7. ☒ Créer repositories, schemas, services (Architecture POO 4 couches)
8. ☐ Télécharger données constellations (Stellarium + D3-Celestial)
9. ☐ Importer constellations dans SQLite

Tâches Frontend :

1. ☒ Setup projet Vite + React + TypeScript
2. ☒ Installer Three.js + React Three Fiber + Drei + Postprocessing
3. ☒ Configurer Axios + Zustand
4. ☐ Test : afficher un cube 3D qui tourne

Livrable semaine 2 :

- ☒ SQLite avec 119k étoiles + 50 points d'observation
 - ☒ Backend FastAPI accessible (<http://localhost:8000/docs>)
 - ☐ Frontend React affichant cube 3D
-

SEMAINES 3-4 : GLOBE 3D INTERACTIF

Objectif : Globe Terre photoréaliste avec points d'observation cliquables

Tâches Backend :

1. Endpoint `GET /api/observation-points` (retourne 50 villes)
2. Tests unitaires service (PyTest)

Tâches Frontend :

1. Télécharger textures NASA (8K Terre jour + night lights + normal map)
2. Compresser textures en WebP
3. Créer composant `<Globe />` avec Three.js :
 - Sphère rayon 1
 - MeshStandardMaterial avec textures PBR
 - Rotation auto lente
 - OrbitControls (rotation manuelle souris)
4. Créer composant `<LocationMarker />` :
 - Pins 3D sur globe
 - Conversion lat/lon → coordonnées 3D sphériques

- OnClick event → highlight marqueur
- 5. Fetch `/api/observation-points` et afficher 50 marqueurs
- 6. Panneau latéral : afficher infos ville sélectionnée
- 7. Effet atmosphère (shader custom bleu clair)

Livrable semaine 4 :

- Globe Terre photoréaliste (8K texture)
 - 50 villes affichées en pins 3D
 - Clic sur ville → highlight + infos affichées
 - Rotation fluide 60 FPS
-

SEMAINES 5-8 : CIEL NOCTURNE (CORE FEATURE)

Objectif : Afficher 5000+ étoiles réalistes au-dessus d'une ville

Tâches Backend :

1. `AstronomyService.compute_visible_stars()` — vectorisé AstroPy/NumPy
2. Endpoint `GET /api/stars/visible` :
 - Paramètres : lat, lon, timestamp, mag_limit
 - Logique :
 1. Pré-filtrage SQL : `SELECT * WHERE magnitude < mag_limit AND dec BETWEEN ...`
 2. Calcul **vectorisé** azimut/altitude (AstroPy)
 3. Filtrer altitude > 0
 4. Retourner JSON
3. Intégration cache en mémoire (TTLCache)
4. Endpoint `GET /api/stars/{id}` (détails étoile)
5. Tests : vérifier Polaris visible depuis Paris (altitude ≈ 48°)

Tâches Frontend :

1. Créer composant `<NightSky />` :
 - BufferGeometry + Points (une draw call pour toutes les étoiles)
 - Couleur basée sur classe spectrale
 - Taille basée sur magnitude (échelle log)
2. Fetch `/api/stars/visible` au clic sur ville
3. Post-processing : Bloom pour étoiles brillantes
4. Raycasting : détection hover/clic étoile
5. Tooltip au hover (nom étoile si connu)
6. Panneau détails au clic étoile
7. Transition caméra animée (globe → ciel)
8. Loading state (spinner pendant calculs)
9. Indicateur points cardinaux (N/S/E/O)

Performance attendue :

- Calcul 5000 étoiles : 300-500ms (avec cache : 50ms)
- Rendu 60 FPS constant

- Mémoire GPU : ~20 Mo

Livable semaine 8 :

- Ciel nocturne avec 5000+ étoiles
 - Couleurs réalistes (types spectraux)
 - Clic étoile → détails complets
 - Bloom post-processing
 - Cache opérationnel
-

SEMAINES 9-10 : CONSTELLATIONS

Objectif : Sélection constellation → navigation automatique + surbrillance

Tâches Backend :

1. Endpoint `GET /api/constellations` (liste 88 constellations)
2. Endpoint `GET /api/constellations/{id}` (nom, étoiles, lignes connexions)
3. Service `find_best_observation_point()` :
 - Pour chaque point, calculer altitude centre constellation
 - Retourner point où altitude maximale
4. Endpoint `GET /api/constellations/{id}/best-location?timestamp=...`
5. Tests : Orion visible hémisphère Nord en hiver

Tâches Frontend :

1. Panneau latéral : liste 88 constellations (search bar)
2. Clic constellation :
 - Fetch best-location → animation caméra vers point optimal
 - Fetch étoiles visibles
 - Afficher ciel avec constellation surbrillancée
3. Composant `<ConstellationPattern />` :
 - Dessiner lignes entre étoiles (Line Three.js)
 - Couleur vive (cyan/jaune)
 - Étoiles pattern agrandies
4. **Slider temporel** pour voyager dans le temps (date/heure arbitraire)
5. Highlight persistant jusqu'à désélection
6. Noms constellations affichés au centre pattern

Livable semaine 10 :

- 88 constellations sélectionnables
 - Navigation automatique vers meilleur point
 - Surbrillance pattern (lignes + étoiles)
 - Slider temporel fonctionnel
 - Transitions animées fluides
-

SEMAINE 11 : POLISH & OPTIMISATIONS

Objectif : Qualité production, UX/UI soignée

Tâches :

1. **Dark theme complet** (noir spatial + accents cyan/violet)
2. **Animations** :
 - Transition globe → ciel (3 secondes smooth)
 - Fade-in étoiles progressif
 - Hover states sur boutons
3. **Loading states** (skeleton loader, progress bar)
4. **Responsive design** (mobile : panneau bottom sheet, touch gestures)
5. **Gestion erreurs** (toast notifications, fallback si API down, retry logic)
6. **Documentation** (docstrings, JSDoc, README)
7. **Tests** :
 - Tests unitaires backend (coverage 70%+, PyTest)
 - Tests composants React (Jest)
8. **Refactoring** : respect SOLID, DRY, typage strict

Ajouts visuels (bonus) :

- Voie lactée texture ESA
- Grille azimut/altitude (toggle)
- Compteur étoiles visibles en temps réel
- Mode jour/nuit sur le globe (termineur dynamique)

Livrable semaine 11 :

- Application polie, fluide, professionnelle
 - Tests passent
 - Documentation complète
-

SEMAINE 12 : DÉPLOIEMENT & PRÉSENTATION

Objectif : Mise en production, démo prête

Tâches Déploiement :

1. **Dockerisation finale** (multi-stage build)
2. **Backend** → **Railway** (deploy depuis GitHub, SSL auto)
3. **Frontend** → **Vercel** (build Vite automatique)
4. **Configuration CORS** production
5. **Tests production** (API accessible, latence < 500ms)

Tâches Présentation :

1. **Vidéo démo** (2-3 minutes)
2. **Slides soutenance** (15-20 slides)
3. **README final** (installation, API docs, crédits)
4. **Rehearsal soutenance** (10-15 min)

Livrable semaine 12 :

- Application en ligne
 - Vidéo démo + slides finalisées
 - Repo GitHub public avec README professionnel
-

9. RÉCAPITULATIF STACK FINALE

Technologies Backend

- **Langage** : Python 3.11
- **Framework** : FastAPI 0.115+
- **Serveur** : Uvicorn (ASGI)
- **Base données** : SQLite (via SQLAlchemy 2.0)
- **Cache** : cachetools.TTLCache (en mémoire)
- **Validation** : Pydantic v2
- **Calculs astro** : AstroPy 7.0 (vectorisé NumPy)
- **Tests** : PyTest + Coverage

Technologies Frontend

- **Framework** : React 18
- **Build** : Vite 5+
- **Langage** : TypeScript
- **3D Engine** : Three.js r160
- **React 3D** : React Three Fiber + Drei + Postprocessing
- **HTTP** : Axios
- **State** : Zustand
- **Tests** : Jest + React Testing Library

Infrastructure

- **Conteneurs** : Docker + Docker Compose
- **Dev DB** : SQLite embarqué
- **Prod Backend** : Railway.app (gratuit)
- **Prod Frontend** : Vercel (gratuit)
- **CI/CD** : GitHub Actions (optionnel)

Données

- **Étoiles** : HYG Database v3.7 (119k étoiles, ODbL)
- **Constellations** : Stellarium + D3-Celestial (GPL/BSD)
- **Textures** : NASA Visible Earth (domaine public)
- **Voie lactée** : ESA Gaia (CC BY-SA 4.0)

TOTAL COÛT : 0€ (100% gratuit en dev ET production)

10. COMMANDES ESSENTIELLES

Installation initiale (semaine 1)

```
# Clone repo (monorepo)
git clone <ton-repo>

# Setup backend
cd backend
python3 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r requirements.txt

# Import données (une seule fois)
python -m scripts.import_data

# Lancer backend
uvicorn app.main:app --reload

# Setup frontend
cd frontend
npm install
npm run dev

# Ou via Docker (tout en un)
docker-compose up
```

Développement quotidien

```
# Sans Docker
cd backend && uvicorn app.main:app --reload
cd frontend && npm run dev

# Avec Docker
docker-compose up
docker-compose logs -f backend
docker-compose down
```

Tests

```
# Backend
cd backend
pytest --cov=app tests/

# Frontend
cd frontend
npm run test
```

11. MÉTRIQUES DE SUCCÈS

Performance :

- ☒ 60 FPS rotation globe + ciel
- ☒ < 2s chargement initial
- ☒ < 500ms affichage 5000 étoiles
- ☒ Cache hit rate > 85%

Fonctionnel :

- ☒ 119 000 étoiles en base
- ☒ 5000+ étoiles visibles simultanément
- ☒ 88 constellations fonctionnelles
- ☒ 50 points d'observation
- ☒ Slider temporel

Qualité code :

- ☒ Coverage tests > 70% backend
- ☒ 0 erreurs TypeScript
- ☒ Architecture POO 4 couches respectée
- ☒ Documentation complète

Déploiement :

- ☒ App accessible publiquement
- ☒ API response time < 200ms (avec cache)

Ce plan détaillé te donne une marche à suivre semaine par semaine. Focus sur le MVP (semaines 1-8), puis constellations (9-10), polish (11), déploiement (12). Tout est gratuit, open source, et optimisé pour performances maximales. 🚀